

Experiment No. - 01

Aim : Installation of MySQL

Software Required :

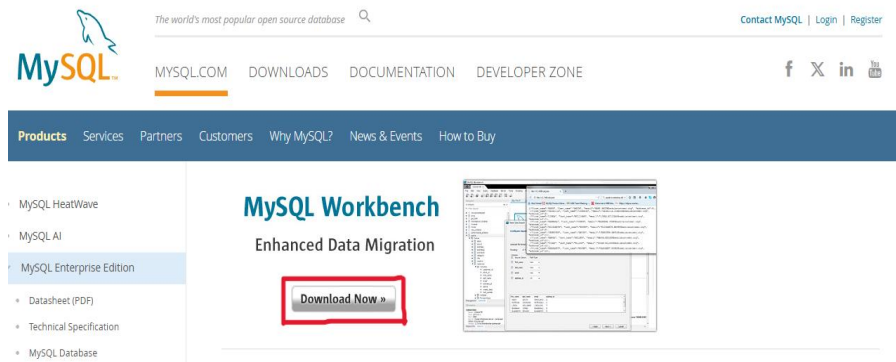
- Windows OS
- Internet Connection
- MySQL Workbench Installer

Procedure :

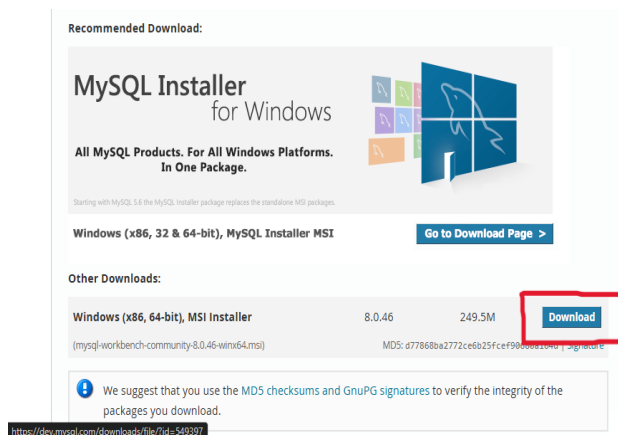
1. Open browser and go to the official MySQL website:

<https://dev.mysql.com/downloads/workbench/>

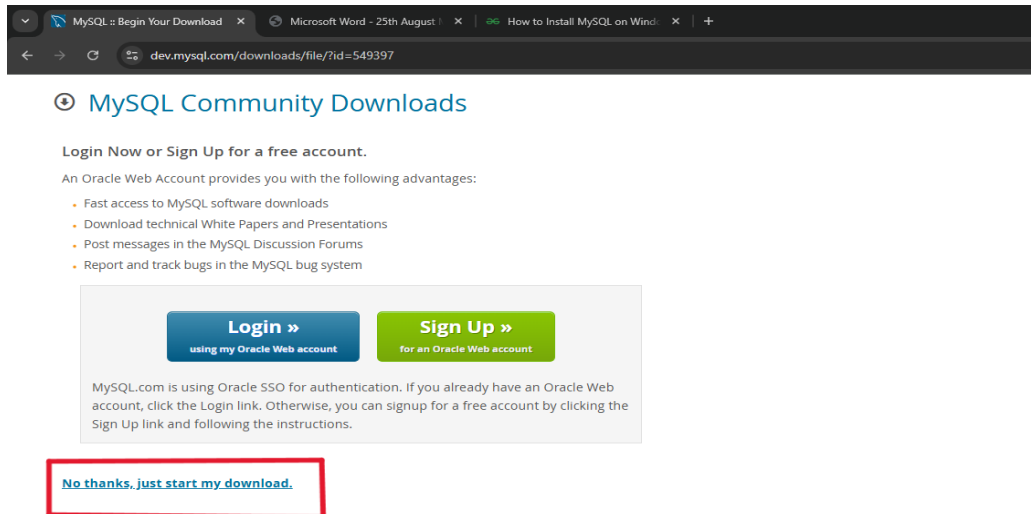
2. Click on **Download Now** button.



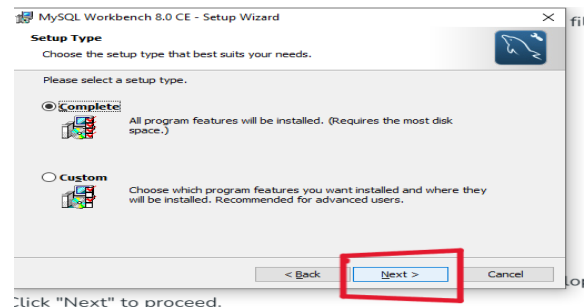
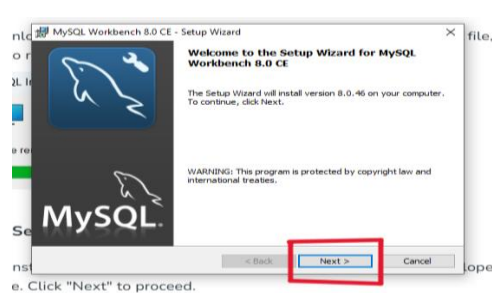
3. Click on **Download** button for Windows (MSI Installer).



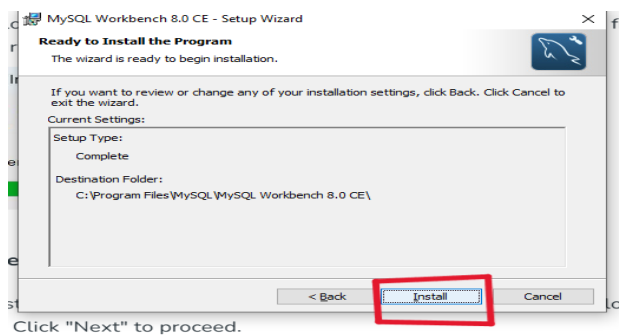
4. Click on **“No thanks, just start my download.”**



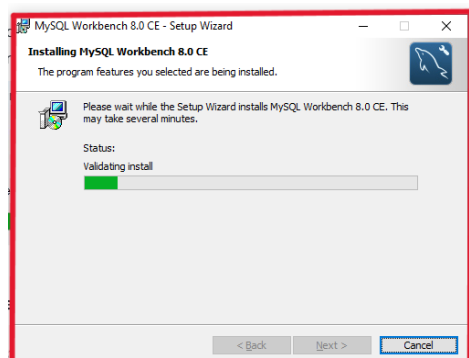
- After download completes, double-click the .msi file.
- Setup Wizard opens → Click **Next**.
- Select **Complete Setup Type** → Click **Next**.



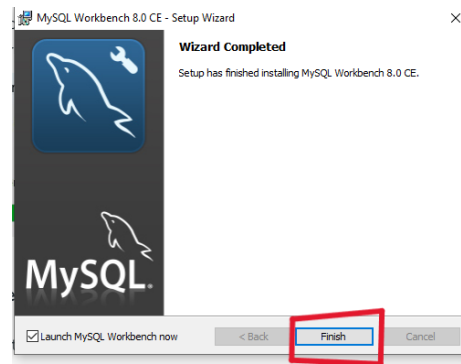
- Click **Install**.



- Wait until installation completes.

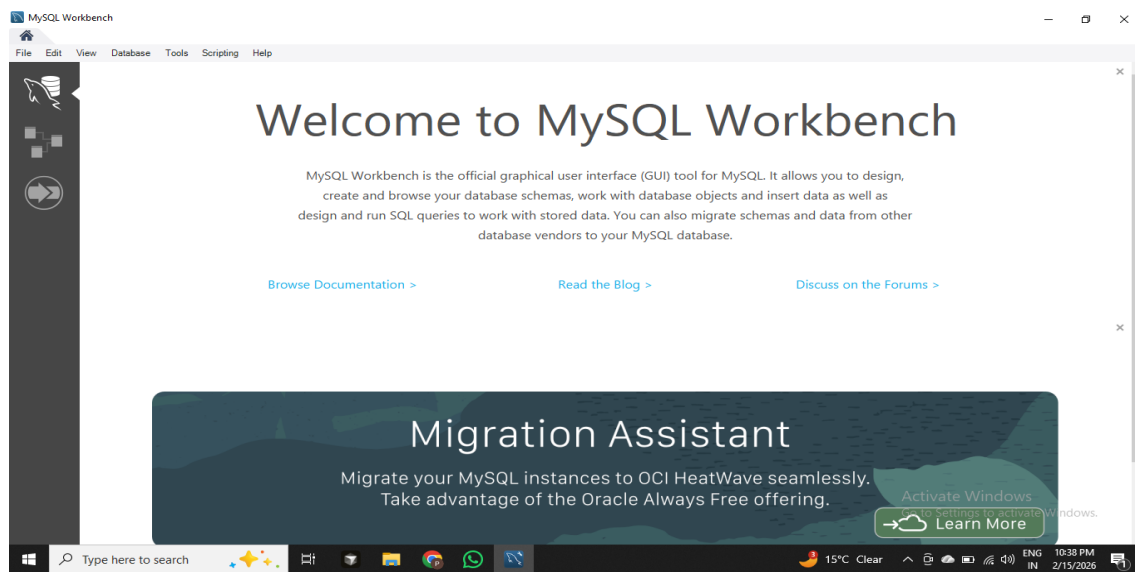


10. Click **Finish**.



Click "Next" to proceed.

11. MySQL Workbench opens successfully.



Result : MySQL Workbench is installed successfully on the system.

Experiment No. - 02

Aim : To study and execute **DDL (Data Definition Language)** commands in SQL

- Create Table
- Modify Table
- Drop Table
- Apply Various Constraints

Software Required :

- Windows OS
- Internet Connection
- MySQL Workbench

Some commonly used DDL commands are:

- **CREATE** – Used to create a new table in the database.
- **ALTER** – Used to change the structure of an existing table.
- **DROP** – Used to delete a table from the database.
- **CONSTRAINTS** – Rules applied to table columns to maintain accuracy and consistency of data.

Types of Constraints

- **PRIMARY KEY** – Identifies each record in a table uniquely.
- **NOT NULL** – Ensures that a column cannot contain NULL values.
- **UNIQUE** – Makes sure that all values in a column are different.
- **FOREIGN KEY** – Creates a relationship between two tables.

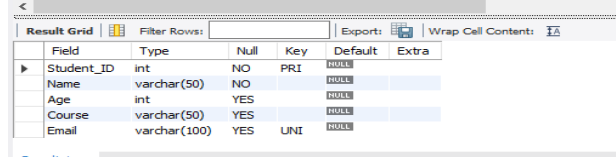
a) Create Table

```
CREATE TABLE Student (  
    Student_ID INT PRIMARY KEY,  
    Name VARCHAR(50) NOT NULL,  
    Age INT CHECK (Age >= 18),  
    Course VARCHAR(50),
```

Email VARCHAR(100) UNIQUE

);

```
3 • CREATE TABLE Student (  
4     Student_ID INT PRIMARY KEY,  
5     Name VARCHAR(50) NOT NULL,  
6     Age INT CHECK (Age >= 18),  
7     Course VARCHAR(50),  
8     Email VARCHAR(100) UNIQUE  
9 )  
10 • DESC Student;
```



Field	Type	Null	Key	Default	Extra
Student_ID	int	NO	PRI	NULL	
Name	varchar(50)	NO		NULL	
Age	int	YES		NULL	
Course	varchar(50)	YES		NULL	
Email	varchar(100)	YES	UNI	NULL	

Explanation :

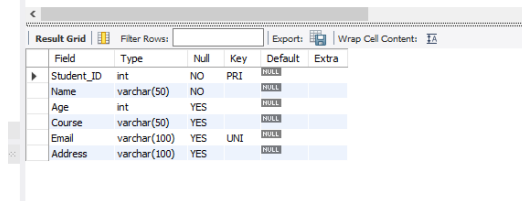
- **Student_ID** acts as the primary key and uniquely identifies each student.
- **Name** cannot be left empty because of the NOT NULL constraint.
- **Age** must be 18 or greater due to the CHECK condition.
- **Email** must contain unique values for every record.

b) Modify Table

1. Add Column

*ALTER TABLE Student
ADD Address VARCHAR(100);*

```
10 • ALTER TABLE Student  
11 ADD Address VARCHAR(100);  
12 • DESC Student;
```



Field	Type	Null	Key	Default	Extra
Student_ID	int	NO	PRI	NULL	
Name	varchar(50)	NO		NULL	
Age	int	YES		NULL	
Course	varchar(50)	YES		NULL	
Email	varchar(100)	YES	UNI	NULL	
Address	varchar(100)	YES		NULL	

2. Modify Column

*ALTER TABLE Student
MODIFY Course VARCHAR(100);*

- This statement increases the size of the **Course** column so that it can store up to **100 characters**.

3. Drop Column

```
ALTER TABLE Student  
DROP COLUMN Address;
```

- This command deletes the **Address** column from the Student table.

c) Applying Constraints

Add Foreign Key to the Student Table

First Create another Table :

```
CREATE TABLE Department (  
    Dept_ID INT PRIMARY KEY,  
    Dept_Name VARCHAR(50)  
);
```

Add foreign key in Student table :

```
ALTER TABLE Student  
ADD Dept_ID INT;  
  
ALTER TABLE Student  
ADD CONSTRAINT FK_Dept  
FOREIGN KEY (Dept_ID)  
REFERENCES Department(Dept_ID);
```

d) Dropping a Table

- This command removes the **Student table along with all its records** from the database.

```
Drop table Student;
```

Result :

The **DDL commands** such as **CREATE**, **ALTER**, and **DROP** were executed successfully and different constraints were applied to the table.

Experiment No. - 03

Aim : Creating Entity-Relationship Diagram using case tools.

Software Required :

- Windows OS
- Internet Connection
- draw.io

Entities and Attributes :

1. Readers :

- user_id (PK)
- name
- email
- contact
- address

2. Books :

- ISBN (PK)
- title
- author_name
- edition
- category
- Price

3. Publishers :

- publisher_id (PK)
- name
- year_of_publishing

4. Staff :

- staff_id (PK)
- Name

5. Issue/Return :

- issue_date
- return_date
- due_date

6. Report :

- book_no
- reg_no
- user_id

7. Authentication System :

- login
- password

Relationships :

1. Readers – Issue_Return – Books

- One Reader can issue many Books.
- One Book can be issued many times.
- Relationship: **Many-to-Many (M:N)**.

2. Books – Publisher

- One Publisher publishes many Books.
- Relationship: **One-to-Many(1:M)**.

3. Readers – Staff

- Staff keeps track of Readers.
- Relationship: **One-to-Many(1:M)**.

4. Staff – Report

- One Staff manages many Reports.
- Relationship: **One-to-Many(1:M)**.

5. Staff – Auth_System

- Each Staff has one Login.
- Relationship: **One-to-One(1:1)**.

6. Staff – Books

- Staff maintains Books.
- Relationship: **One-to-Many(1:M)**.

E- R Diagram :



Result : Entity-Relationship Diagram for Library Management System was successfully created using CASE tool.

Experiment No. - 04

Aim : To understand and execute **DDL and DML commands in SQL**.

- Create a table with constraints
- Insert records into the table
- Modify existing data
- Delete records from the table

Software Required :

- Windows Operating System
- Internet Connection
- MySQL Workbench

> DDL (Data Definition Language):

DDL commands are used to **define or modify the structure of database objects** like tables. Example: CREATE, ALTER, DROP.

> DML (Data Manipulation Language):

DML commands are used to **manipulate the data stored in tables**.
Example: INSERT, UPDATE, DELETE.

1) Create Table with specified constraints

- This command creates a **Student table** with constraints like **Primary Key, Not Null, Check, and Unique**.

```
CREATE TABLE Student (  
    Student_ID INT PRIMARY KEY,  
    Name VARCHAR(50) NOT NULL,  
    Age INT CHECK (Age >= 18),  
    Course VARCHAR(50),  
    Email VARCHAR(100) UNIQUE
```

);

2. Insert data

- This
into the

INSERT

Age,

VALUES (101, 'Payal', 22, 'MCA', 'payal@gmail.com'),

(102, 'Akanksha', 21, 'MSc', 'ak@gmail.com');

```
10 • INSERT INTO Student (Student_ID, Name, Age, Course, Email)
11 VALUES
12 (101, 'Payal', 22, 'MCA', 'payal@gmail.com'),
13 (102, 'Akanksha', 21, 'MSc', 'ak@gmail.com');
14 • Select * from Student;
```

Student_ID	Name	Age	Course	Email
101	Payal	22	MCA	payal@gmail.com
102	Akanksha	21	MSc	ak@gmail.com

into table

command inserts a **new record**
Student table.

INTO Student (Student_ID, Name,
Course, Email)

3. Update Data of Table

- This command updates the course of the student whose Student_ID is 101.

UPDATE Student SET Course = 'BCA' WHERE Student_ID = 101;

```
14 • UPDATE Student SET Course = 'BCA' WHERE Student_ID = 101;
15 • Select * from Student;
```

Student_ID	Name	Age	Course	Email
101	Payal	22	BCA	payal@gmail.com
102	Akanksha	21	MSc	ak@gmail.com

4. Delete rows from table → This command removes the row where Student_ID is 101 from the Student table.

DELETE FROM Student WHERE Student_ID = 101;

```
15 • DELETE FROM Student WHERE Student_ID = 101;
16 • Select * from Student;
```

Student_ID	Name	Age	Course	Email
102	Akanksha	21	MSc	ak@gmail.com

Result :

Thus, **DDL and DML commands** were executed successfully to create, insert, update, and delete data in a table.

Experiment No. - 05

Aim : To write SQL statements using MY SQL:

- Basic SQL SELECT statements
- Restricting and sorting data

→ The **SELECT statement** is used to retrieve data from a database table. It allows users to view specific columns, apply conditions, sort results, and combine data from multiple tables.

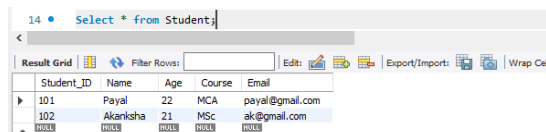
Software Required :

- Windows OS
- Internet Connection
- MySQL Workbench

a) Writing basic SQL SELECT statements

→ This command displays **all columns and all rows** from the Student table.

```
SELECT * FROM Student;
```

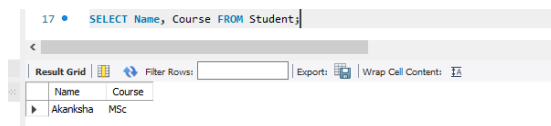


The screenshot shows the MySQL Workbench interface. The SQL editor at the top contains the query `SELECT * FROM Student;`. Below the editor, the 'Result Grid' tab is active, displaying a table with 5 columns: Student_ID, Name, Age, Course, and Email. There are two rows of data.

Student_ID	Name	Age	Course	Email
101	Payal	22	MCA	payal@gmail.com
102	Akanksha	21	MSc	ak@gmail.com

→ This command displays only the **Name and Course columns** from the Student table.

```
SELECT Name, Course FROM Student;
```



The screenshot shows the MySQL Workbench interface. The SQL editor at the top contains the query `SELECT Name, Course FROM Student;`. Below the editor, the 'Result Grid' tab is active, displaying a table with 2 columns: Name and Course. There is one row of data.

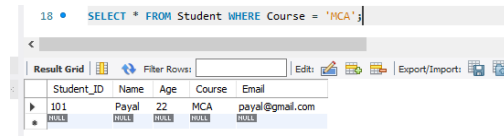
Name	Course
Akanksha	MSc

b) Restricting and Sorting Data

Restricting Data (using WHERE) :

→ This command displays only those students **whose course is MCA**.

SELECT * FROM Student WHERE Course = 'MCA';



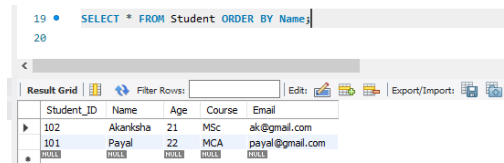
The screenshot shows a SQL query editor with the command `SELECT * FROM Student WHERE Course = 'MCA';`. Below the editor, a 'Result Grid' displays the query results. The grid has columns for Student_ID, Name, Age, Course, and Email. One record is shown: Student_ID 101, Name Payal, Age 22, Course MCA, and Email payal@gmail.com.

Student_ID	Name	Age	Course	Email
101	Payal	22	MCA	payal@gmail.com

Sorting Data (using ORDER BY) :

→ This command displays the student records sorted in ascending order by Name.

SELECT * FROM Student ORDER By Name;



The screenshot shows a SQL query editor with the command `SELECT * FROM Student ORDER BY Name;`. Below the editor, a 'Result Grid' displays the query results. The grid has columns for Student_ID, Name, Age, Course, and Email. Two records are shown, sorted by name: Student_ID 102, Name Akanksha, Age 21, Course MSc, and Email ak@gmail.com; and Student_ID 101, Name Payal, Age 22, Course MCA, and Email payal@gmail.com.

Student_ID	Name	Age	Course	Email
102	Akanksha	21	MSc	ak@gmail.com
101	Payal	22	MCA	payal@gmail.com

c) Displaying data from multiple tables

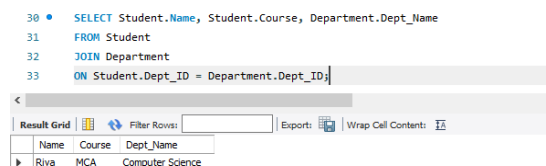
→ This command displays data from both Student and Department tables by joining them using Dept_ID.

First create another table :

```
CREATE TABLE Department (  
    Dept_ID INT PRIMARY KEY,  
    Dept_Name VARCHAR(50)  
);
```

Join Example :

```
SELECT Student.Name, Student.Course, Department.Dept_Name  
FROM Student  
JOIN Department  
ON Student.Dept_ID = Department.Dept_ID;
```



The screenshot shows a SQL query editor with the following command: `SELECT Student.Name, Student.Course, Department.Dept_Name FROM Student JOIN Department ON Student.Dept_ID = Department.Dept_ID;`. Below the editor, a 'Result Grid' displays the query results. The grid has columns for Name, Course, and Dept_Name. One record is shown: Name Riya, Course MCA, and Dept_Name Computer Science.

Name	Course	Dept_Name
Riya	MCA	Computer Science

Experiment No. - 06

Aim : To study and execute SQL commands for

- Retrieve the information from the table(Basic Queries)
- Aggregate functions
- Retrieval by character matching

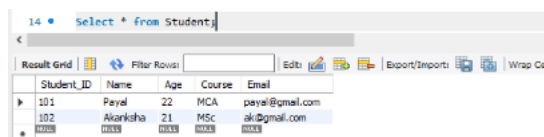
Software Required :

- Windows Operating System
- Internet Connection
- MySQL Workbench

1. Retrieve Information from a Table (Basic Queries)

- This query displays **all the columns and records present in the Student table.**

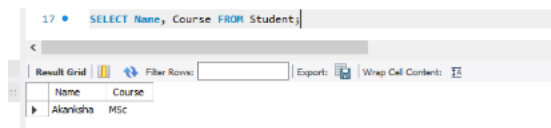
```
SELECT * FROM Student;
```



Student_ID	Name	Age	Course	Email
101	Payal	22	MCA	payal@gmail.com
102	Akanksha	21	MSc	ak@gmail.com

- This query displays only the **Name and Course fields** from the Student table.

```
SELECT Name, Course FROM Student;
```



Name	Course
Akanksha	MSc

2. Aggregate Functions

- Aggregate functions are used to **perform calculations on a set of rows and return a single result.**

Count :

- This function returns the **total number of records in the Student table.**

```
SELECT COUNT(*) FROM Student;
```

34 • `SELECT COUNT(*) FROM Student;`

COUNT(*)
3

Average :

- This function calculates the **average value of the Age column**.

`SELECT AVG(Age) FROM Student;`

35 • `SELECT AVG(Age) FROM Student;`

AVG(Age)
21.6667

3. Retrieval Using Character Matching

- Character matching is performed using the **LIKE operator**, which searches for specific patterns in text.

`SELECT * FROM Student WHERE Name LIKE 'P%';`

36 • `SELECT * FROM Student WHERE Name LIKE 'P%';`

Student_ID	Name	Age	Course	Email	Dept_ID
101	Payal	22	MCA	payal@gmail.com	1001
102	Payal	22	MCA	payal@gmail.com	1001

Result :

The SQL queries were successfully executed to **retrieve data from the table, perform aggregate calculations, and search records using character matching**.